C# GUI – WinForms / alap WPF nagyjegyzet

Részletes tételvázlat, felépített eseménykezelés, bemásolható mintaalkalmazások

• WinForms projektindítás és formok
• vezérlők, események, validáció
• DataGridView, fájlkezelés, dialógusok
• OOP-s szervezés GUI-ban
• Timer, async, UX és mini projektek

Megjegyzés: a dokumentum nem a teljes C# nyelv minden létező részletét tartalmazza, hanem egy nagyon széles, feladatmegoldásra optimalizált, gyakorlati nagyjegyzetet.

Tartalomjegyzék

1. 1. GUI szemlélet és eseményvezérelt működés
2. 2. WinForms projekt alapok
3. 3. Vezérlők és tulajdonságok
4. 4. Eseménykezelés
5. 5. Input validáció
6. 6. Listák, táblák, DataGridView
7. 7. Fájlkezelés GUI-ból
8. 8. Több ablak, dialógusok
9. 9. OOP a felületi alkalmazásokban
10. 10. Aszinkronitás és Timer
11. 11. Hibakezelés és UX
12. 12. Teljes mini projektek
13. 13. WPF alap összehasonlítás

1. GUI szemlélet és eseményvezérelt működés

A GUI-s C# fejlesztésben nem lineárisan fut a program, hanem események indítanak el műveleteket.

- A felhasználó kattint, gépel, választ; ezek eseményeket váltanak ki.
- A vezérlők állapotát kódból olvasod és írod.
- A logikát érdemes külön metódusokba vagy külön osztályokba szervezni.
- A GUI feladatoknál a validáció és a hibaüzenet ugyanolyan fontos, mint maga a számítás.

GUI gondolkodásmód

```
// Tipikus eseménylánc:
// 1. Felhasználó beír két számot TextBox-ba
// 2. Button kattintás történik
// 3. Kódban beolvassuk a szövegeket
// 4. Ellenőrizzük, hogy számok-e
// 5. Kiszámoljuk az eredményt
// 6. Kiírjuk Label-be vagy MessageBox-ba
```

2. WinForms projekt alapok

- A főablak tipikusan Form1 néven indul.
- A Designer felületen lerakod a vezérlőket.
- A háttérben .cs fájlban írod az eseménykezelő kódot.

Alap WinForms szerkezet

```
namespace GuiPelda
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

3. Vezérlők és tulajdonságok

Vezérlő	**Feladata**
Label	Szöveg megjelenítése.
TextBox	Szöveg bevitele.
Button	Művelet indítása kattintásra.
CheckBox	Igaz/hamis jellegű választás.
RadioButton	Egymást kizáró opciók.
ComboBox	Legördülő lista.
ListBox	Listaelemek megjelenítése.

DataGridView	Táblázatos adatok.
PictureBox	Kép megjelenítése.
ProgressBar	Folyamat kijelzése.

Gyakori tulajdonságok

```
// Fontos tulajdonságok példák
label1.Text = "Kész";
textBox1.Text = "alma";
button1.Enabled = true;
checkBox1.Checked = false;
comboBox1.SelectedIndex = 0;
listBox1.Items.Add("Elem 1");
```

4. Eseménykezelés

Button Click esemény

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("A gomb működik!");
}
```

TextChanged esemény

```
private void textBox1_TextChanged(object sender, EventArgs e)
{
    labelKarakterDb.Text = textBox1.Text.Length.ToString();
}
```

ComboBox esemény

```
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    MessageBox.Show($"Kiválasztva: {comboBox1.SelectedItem}");
}
```

Névkonvenció segít: txtNev, btnMent, lblStatus, dgvAdatok stb.

5. Input validáció

Alap validációs metódus

```
private bool AdatokEllenorzese()
{
    if (string.IsNullOrWhiteSpace(txtNev.Text))
    {
        MessageBox.Show("A név megadása kötelező.");
        txtNev.Focus();
        return false;
    }

    if (!int.TryParse(txtKor.Text, out int kor) || kor < 0 || kor > 130)
    {
        MessageBox.Show("Adj meg érvényes életkort.");
        txtKor.Focus();
        return false;
```

```
    }

    return true;
}
```

Validáció használata kattintáskor

```
private void btnMent_Click(object sender, EventArgs e)
{
    if (!AdatokEllenorzese())
        return;

    MessageBox.Show("Sikeres mentés.");
}
```

6. Listák, táblák, DataGridView

ListBox feltöltése

```
private void Form1_Load(object sender, EventArgs e)
{
    listBox1.Items.Add("alma");
    listBox1.Items.Add("körte");
    listBox1.Items.Add("szilva");
}
```

ComboBox feltöltése

```
comboBox1.Items.Clear();
comboBox1.Items.Add("Kezdő");
comboBox1.Items.Add("Haladó");
comboBox1.Items.Add("Profi");
comboBox1.SelectedIndex = 0;
```

DataGridView kézi feltöltése

```
private void Form1_Load(object sender, EventArgs e)
{
    dgvAdatok.ColumnCount = 3;
    dgvAdatok.Columns[0].Name = "Név";
    dgvAdatok.Columns[1].Name = "Kor";
    dgvAdatok.Columns[2].Name = "Város";

    dgvAdatok.Rows.Add("Anna", 20, "Szeged");
    dgvAdatok.Rows.Add("Béla", 24, "Kecskemét");
}
```

Új sor hozzáadása

```
private void btnSorHozzaad_Click(object sender, EventArgs e)
{
    dgvAdatok.Rows.Add(txtNev.Text, txtKor.Text, txtVaros.Text);
}
```

7. Fájlkezelés GUI-ból

TextBox tartalom mentése és betöltése

```
using System.IO;

private void btnMentFajlba_Click(object sender, EventArgs e)
{
    File.WriteAllText("adat.txt", txtTartalom.Text);
    MessageBox.Show("Mentve.");
}

private void btnBetolt_Click(object sender, EventArgs e)
{
    if (File.Exists("adat.txt"))
        txtTartalom.Text = File.ReadAllText("adat.txt");
}
```

OpenFileDialog használata

```
private void btnMegnyit_Click(object sender, EventArgs e)
{
    OpenFileDialog ofd = new OpenFileDialog();
    ofd.Filter = "Szövegfájlok|*.txt|Minden fájl|*.*";

    if (ofd.ShowDialog() == DialogResult.OK)
    {
        txtTartalom.Text = File.ReadAllText(ofd.FileName);
    }
}
```

SaveFileDialog használata

```
private void btnMentesMas_Click(object sender, EventArgs e)
{
    SaveFileDialog sfd = new SaveFileDialog();
    sfd.Filter = "Szövegfájlok|*.txt";

    if (sfd.ShowDialog() == DialogResult.OK)
    {
        File.WriteAllText(sfd.FileName, txtTartalom.Text);
    }
}
```

8. Több ablak, dialógusok

Második ablak megnyitása

```
private void btnUjAblak_Click(object sender, EventArgs e)
{
    Form2 ablak = new Form2();
    ablak.Show();
}
```

Modális ablak

```
private void btnDialogus_Click(object sender, EventArgs e)
{
    using Form2 ablak = new Form2();
```

```
    if (ablak.ShowDialog() == DialogResult.OK)
    {
        MessageBox.Show("A párbeszédablak OK-val zárult.");
    }
}
```

9. OOP a felületi alkalmazásokban

Adatosztály GUI-hoz

```
public class Diak
{
    public string Nev { get; set; }
    public int Pont { get; set; }

    public Diak(string nev, int pont)
    {
        Nev = nev;
        Pont = pont;
    }

    public override string ToString()
    {
        return $"{Nev} - {Pont} pont";
    }
}
```

Lista + osztály + frissítés

```
private List<Diak> diakok = new List<Diak>();

private void btnHozzaad_Click(object sender, EventArgs e)
{
    if (!int.TryParse(txtPont.Text, out int pont))
    {
        MessageBox.Show("Pont csak szám lehet.");
        return;
    }

    Diak uj = new Diak(txtNev.Text, pont);
    diakok.Add(uj);
    FrissitLista();
}

private void FrissitLista()
{
    listBox1.Items.Clear();
    foreach (Diak d in diakok)
        listBox1.Items.Add(d);
}
```

10. Aszinkronitás és Timer

Timer példa

```
private System.Windows.Forms.Timer idozo = new System.Windows.Forms.Timer();
```

```
private void Form1_Load(object sender, EventArgs e)
{
    idozo.Interval = 1000;
    idozo.Tick += Idozo_Tick;
    idozo.Start();
}

private void Idozo_Tick(object? sender, EventArgs e)
{
    lblOra.Text = DateTime.Now.ToString("HH:mm:ss");
}
```

async eseménykezelő

```
private async void btnLetoltes_Click(object sender, EventArgs e)
{
    btnLetoltes.Enabled = false;
    lblStatus.Text = "Betöltés folyamatban...";

    await Task.Delay(2000); // szimulált hosszú művelet

    lblStatus.Text = "Kész";
    btnLetoltes.Enabled = true;
}
```

Hosszú műveletet ne a UI szálon futtass blokkoló módon, mert befagy a felület.

11. Hibakezelés és UX

Hibakezelés GUI-ban

```
try
{
    int a = int.Parse(txtA.Text);
    int b = int.Parse(txtB.Text);
    lblEredmeny.Text = (a / b).ToString();
}
catch (FormatException)
{
    MessageBox.Show("Csak számot adj meg.");
}
catch (DivideByZeroException)
{
    MessageBox.Show("Nullával nem oszthatsz.");
}
catch (Exception ex)
{
    MessageBox.Show("Váratlan hiba: " + ex.Message);
}
```

- A felhasználó mindig kapjon érthető visszajelzést.
- A hibás mezőre érdemes Focus()-t adni.
- Sikeres mentésnél is jelezz vissza.
- A gombok állapotát vezesd: mentés közben tiltás, végén újra engedélyezés.

12. Teljes mini projektek

Mini projekt 1 – Egyszerű számológép

```
private void btnOsszead_Click(object sender, EventArgs e)
{
    if (double.TryParse(txtElso.Text, out double a) &&
        double.TryParse(txtMasodik.Text, out double b))
    {
        lblEredmeny.Text = (a + b).ToString();
    }
    else
    {
        MessageBox.Show("Érvényes számokat adj meg.");
    }
}
```

Mini projekt 2 – Tanulólista mentéssel

```
private void btnMentLista_Click(object sender, EventArgs e)
{
    using StreamWriter sw = new StreamWriter("diakok.txt");
    foreach (Diak d in diakok)
    {
        sw.WriteLine($"{d.Nev};{d.Pont}");
    }
}
```

Mini projekt 3 – Képfeltöltő PictureBox-szal

```
private void btnKepBetolt_Click(object sender, EventArgs e)
{
    OpenFileDialog ofd = new OpenFileDialog();
    ofd.Filter = "Képek|*.jpg;*.jpeg;*.png;*.bmp";

    if (ofd.ShowDialog() == DialogResult.OK)
    {
        pictureBox1.Image = Image.FromFile(ofd.FileName);
        pictureBox1.SizeMode = PictureBoxSizeMode.Zoom;
    }
}
```

13. WPF alap összehasonlítás

Technológia	Jellemzés
WinForms	Gyors, vizuális, kezdőknek és iskolai feladatokra nagyon jó.
WPF	Erősebb adatbinding, modernebb felépítés, XAML alapú UI.

Nagyon rövid WPF ízelítő

```
<!-- WPF XAML példa -->
<StackPanel Margin="20">
    <TextBox x:Name="txtNev" Margin="0,0,0,10" />
    <Button Content="Kattints" Click="Button_Click" />
</StackPanel>
```

WPF eseménykezelő példa

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Szia " + txtNev.Text);
}
```

Ha iskolai GUI feladatokról van szó, a WinForms minták a leggyakrabban azonnal használhatók.